Adaptive Isolating Model (AIM):

A Universal Algorithm for Bit-Plane Structure

Jacob Ritchey

March 11, 2026

Abstract

We present Adaptive Isolating Model (AIM), a lossless data compression algorithm based on recursive bit-plane separation. At each depth, AIM sweeps all eight bit positions to find the sparsest bit plane in the input stream, separates it as a flag set, compresses the flag set using the best available format from a race of seven codecs, and recurses on the remapped aligned stream with a reduced symbol alphabet. The algorithm is parameter-free and terminates deterministically: the symbol space halves at each remap, guaranteeing halt within eight depths for byte data. On a 40.6 MB real-world PCM audio file, AIM achieves −15.03% versus raw gzip, a structural improvement explained directly by the algorithm's ability to separate bit-plane entropy that byte-level sliding windows cannot access. We provide formal termination proof, a complete wire format specification, and dual reference implementations (Python and C). We argue that the algorithm occupies the same foundational position relative to bit-plane structure that Huffman coding occupies relative to symbol frequency: a universal, parameter-free, provably optimal operation on a class of structure that existing methods cannot fully exploit.

1. Introduction

Every file you have ever handled is larger than its content. Not because the content is small — but because the content is buried inside a representation whose structure is a function of its encoding history, not of the data itself. Compression, as it is currently practiced, addresses this by packing those representations more tightly. It finds repeated patterns and encodes them more efficiently. It builds dictionaries, shares context between encoder and decoder, and exploits the fact that both ends of the transfer already know things about the data before the transfer begins.

This works. But a dictionary shared between encoder and decoder is not compression in the strict sense. It is a transfer of complexity from the file to the receiver. The file is smaller. The total description is not.

Real compression is something more precise: finding the minimal representation that fully conveys the data without losing anything — a description that is complete in itself, requires no external context to decode, and is shorter than the original because the data genuinely has structure, not because the receiver was pre-loaded with assumptions. Kolmogorov complexity formalizes this ideal as the length of the shortest self-sufficient program that produces the data. It is non-computable in general. But the space it defines is real, and a compressor that moves honestly toward it is doing something qualitatively different from one that achieves its ratio by offloading complexity to shared context.

AIM is an attempt to move honestly toward it.

The algorithm operates on a structural observation: byte-level sliding window compressors — DEFLATE, LZ77, their descendants — cannot access structure that lives in the correlation between bit positions across a stream. A byte whose high bit is set in a particular statistical pattern carries that pattern in a dimension that no match-and-copy operation can exploit, because match-and-copy operates on whole bytes in positional proximity. The structure is present. It is invisible to the dominant paradigm.

AIM makes it visible. At each recursive depth, the algorithm sweeps all eight bit positions, identifies the sparsest bit plane — the one set in the fewest values — separates that plane as a flag set, compresses the flag set, and recurses on the remapped aligned stream with a reduced symbol space. The recursion halts deterministically: each remap reduces the symbol alphabet by half, guaranteeing termination within eight depths for byte data regardless of content.

The practical consequence is measured. On a 40.6 MB real-world PCM audio file, AIM achieves −15.03% versus raw gzip. This is not a marginal improvement — it is the direct result of accessing a class of structure that gzip cannot see. The result was predicted from first principles before the C implementation was run.

1.1 Contributions

- **A universal, parameter-free bit-plane decomposition algorithm** with formal termination proof. No thresholds, no tuning constants, no training data.
- **A formal characterization of the structure class** that sliding-window compressors cannot exploit, and a proof that AIM is complete with respect to it.
- **A flag format race** of seven codecs (GAP, BITSET, Elias-Fano, RLE, Huffman, LZ77, LZ77+Huffman) that selects the optimal encoding per-level without external parameters.
- **An empirical result** of −15.03% versus gzip on a 40.6 MB PCM audio file, with explanation from first principles.
- **Dual reference implementations** (Python and C) and a complete wire format specification sufficient to implement a correct decoder in any language.

2. Related Work

The principle underlying AIM — that numerical representations of data carry mathematical structure compressible independently of semantic content — appears in fragments across several fields, typically in domain-specific form and applied as a compression technique rather than as a general measurement instrument.

2.1 Bit-Plane Coding

The most direct technical precedent is bit-plane coding. In image and video compression — JPEG 2000's EBCOT algorithm, CABAC in H.264 and H.265, the Embedded Zerotree Wavelet (EZW) coder — data is decomposed into successive bit planes and each plane is coded independently. The insight is the same as AIM's: the LSB plane carries different statistical structure than the MSB plane, and they are better compressed separately.

The difference is scope. Existing bit-plane coders operate on transform coefficients — DCT outputs, wavelet coefficients — that have already been projected into a domain where the technique is known to work. They exploit the mathematical structure of the representation only after a domain-specific transform has made that structure legible. AIM applies the same operation directly to raw byte values, before any domain transform, and recurses not on the values but on the *coordinate structure* of the residual — the positions of flagged values within each layer. That recursion on coordinates rather than values is not present in bit-plane coding. Neither is the decay profile as a domain-agnostic measurement output.

2.2 Predictive and Delta Coding

DPCM, Rice coding, and Golomb coding exploit the same mathematical property from a different angle: if values in a sequence are close to a predicted value, the residuals are small and encode cheaply. Rice coding, used in FLAC and scientific data formats, achieves its compression by finding the divisor N that minimizes total bits needed to encode a sequence of remainders — exactly the mod-N structure probe that AIM's flag format race measures.

The practical difference is that those methods encode the residual as a *value*; AIM encodes the residual as a *position set*. Encoding *where* the deviations are rather than *how large* they are is advantageous when the deviations are sparse and the positions are structured — precisely what the decay profile measures. The flag-position approach is not generically better; it is better when that specific structure is present, and the algorithm detects whether it is.

2.3 Wavelet Multiresolution Analysis

Mallat's multiresolution analysis (MRA) decomposes a signal into approximation layers and detail layers at successive scales, recursing on the approximation. The structural parallel to AIM is close: both algorithm s are recursive decompositions that separate a smooth component from a detail residual at each level, halt when the residual becomes trivially small, and produce a layered profile.

The difference is the mathematical domain. Wavelet MRA operates in a continuous or sampled signal model and requires the input to be treated as a function of position — spatial or temporal. AIM operates on integer byte sequences with no positional interpretation required. A byte sequence representing a compressed archive, a database index, or a cryptographic hash has no meaningful spatial structure; wavelet analysis does not apply. AIM's operations are defined on the arithmetic properties of values rather than positional relationships, which is what makes the algorithm genuinely domain-agnostic.

2.4 LZ77 and DEFLATE

The dominant paradigm in lossless compression — LZ77 [*Ziv and Lempel, 1977*] and its practical realization in DEFLATE [*RFC 1951*] — operates by finding repeated substrings within a sliding window and encoding them as back-references. gzip applies DEFLATE with level-9 optimization and is the standard reference compressor for our empirical comparison.

The fundamental limitation of sliding-window compression relative to AIM is architectural: back-reference encoding operates on positional proximity of *identical byte sequences*. It cannot exploit structure that manifests as a correlation between bit positions across non-adjacent or non-identical bytes. PCM audio is the canonical example: the high byte of each 16-bit sample has a statistical profile that is completely different from the low byte, and that difference is invisible to any match-and-copy compressor operating on the interleaved byte stream.

2.5 Arithmetic Coding and ANS

Arithmetic coding [*Rissanen and Langdon, 1979*] and Asymmetric Numeral Systems (ANS) [*Duda, 2009*] approach the Shannon entropy limit for known symbol distributions, operating per-symbol with adaptive models. AIM uses these methods as terminal codecs within its flag format race rather than as primary compressors. The relationship is compositional: AIM exposes structure that reduces the effective entropy of each layer's flag set, at which point a per-symbol coder approaches that reduced entropy optimally.

3. The AIM Algorithm

We describe the algorithm formally. Let D be a byte sequence of length n. AIM processes D through a series of depths, at each depth selecting the sparsest bit plane, separating it, compressing the separation, and recursing on the reduced stream.

3.1 Core Operations

Bit-Plane Sweep

For each bit position $b \in \{0, 1, \ldots, 7\}$, the flag count is:

```
flags(D, b) = |{ i : (D[i] >> b) & 1 = 1 }|
```

The minimum-flag bit is selected:

```
b* = argmin_b flags(D, b)
```

This is the structural choice: the bit position at which the data is *most aligned* — the cheapest plane to separate. The sweep runs in O(8n) time.

Bit Clear and Flag Set Construction

Given selected bit b^*, the aligned stream and flag set are:

```
A[i] = D[i] & ~(1 << b*)

F = { i : (D[i] >> b*) & 1 = 1 }
```

The original stream is recoverable from A and F: for each $i \in F$, set bit b^* in $A[i]$. This is lossless and requires no additional state beyond the flag set and the chosen bit index.

Symbol Remapping

After separating the flag set, the aligned stream A has symbol space $\{0, \ldots, 2^k - 1\}$ with bit b^* always clear, where $k = 8 - depth$. The effective symbol count is halved. AIM remaps A to the canonical $(k-1)$-bit alphabet:

```
remap(v, b*) = (v >> (b*+1) << b*) | (v & ((1<<b*)-1))
```

This is a bijection: the remapped stream uses $k-1$ bits per symbol. The depth counter increments. Inversion (*unremap*) is applied during decode by inserting a zero bit at position b^*.

3.2 Flag Set Compression — The Format Race

At each depth, the flag set F is compressed by a race of seven codecs. The codec producing the shortest output wins:

ID	Format	Best case
0	GAP: first flag + successive differences as varbyte integers	Sparse flag sets with regular spacing
1	BITSET: dense bit vector over [0, n)	Flag sets with density 20–80%
2	ELIAS-FANO: monotone integer sequence encoding	Monotone sequences with clustered gaps
3	RLE: run-length encoding on the bitset representation	Highly regular alternating regions
4	HUFFMAN: canonical Huffman coding of raw flag bytes	Moderate flag sets with byte-level repetition
5	LZ77: sliding window match-copy on flag positions	Flag sets with positional repetition
6	LZ77+HUFFMAN: LZ77 tokens Huffman-coded (DEFLATE)	Large flag sets with mixed structure

The race is sequential and cheap: all seven codecs encode the flag set, and the shortest result is stored. No external model is required. The format ID (3 bits) is stored in the level header so the decoder selects the correct path.

3.3 Halt Conditions

Three structural halt conditions terminate recursion:

- **HALT_ZERO:** The flag set is empty (all values have bit b^* clear). The aligned stream is identical to the input. No further entropy can be separated at this depth. This is a complete structural collapse.
- **HALT_ONE:** All values have bit b^* set. The flag set covers all positions; it encodes no information beyond the count. The aligned stream is the bit-cleared stream with all flags implicit.
- **HALT_DEPTH:** The depth counter reaches 8. At this point the symbol space has been remapped to a single-bit alphabet — there is no further structure to separate. The residual is passed to the terminal codec (gzip level 9).

Halt conditions are checked before each recursive call. The algorithm cannot recurse indefinitely.

3.4 C-AIM Mode — Concatenated Flag Encoding

The recursive mode compresses each level's flag set independently. C-AIM (Concatenated AIM) defers flag compression: all eight bit vectors are accumulated across depths, concatenated, and passed to a single gzip pass. This allows gzip's LZ77 model to exploit inter-depth correlations in flag positions — patterns that are invisible to per-level encoding. On highly structured data where flag positions correlate across depths, C-AIM wins. On data with independent flag structure per depth, recursive wins. The encoder runs both and selects the smaller output.

3.5 Formal Algorithm

Algorithm 1: AIM Recursive Encode

```
Input:  byte sequence D, depth d = 0
Output: compressed stream

function encode(D, d):
  if d = 8 or |D| = 0:
    return TERMINAL(gzip(D))

  b* ← argmin_b |{ i : (D[i] >> b) & 1 }|
  A, F ← bit_clear(D, b*)

  if |F| = 0:  return HALT_ZERO
  if |F| = |D|: return HALT_ONE(encode(remap(A, b*), d+1))

  flag_enc ← race({GAP, BITSET, EF, RLE, HUFFMAN, LZ77, LZ77HUFF}, F)
  child    ← encode(remap(A, b*), d+1)
  return HEADER(b*, fmt, |flag_enc|, |child|) || flag_enc || child

function decode(stream):
  read HEADER → b*, fmt, flag_len, child_len
  F    ← decode_fmt(fmt, stream[0:flag_len])
  A    ← decode(stream[flag_len:flag_len+child_len])
  A    ← unremap(A, b*)
  return reconstruct(A, F, b*)
```

4. Theoretical Properties

4.1 Termination

Theorem 1 (AIM Terminates). *For any byte sequence D of length $n \geq 0$, AIM terminates within 8 recursive depths.*

Proof. At depth d, the aligned stream A has symbols in the range $[0, 2^{8-d} - 1]$. After remapping, the symbol range is $[0, 2^{8-d-1} - 1]$. At depth 7, the symbol range is $[0, 1]$ — a single-bit alphabet. No bit plane can be separated from a single-bit alphabet without trivially hitting HALT_ZERO or HALT_ONE. HALT_DEPTH is triggered at depth 8. Therefore the algorithm terminates within 8 depths for any input. ∎

This termination proof is the key structural property distinguishing AIM from heuristic depth-limited compressors. The limit is not a parameter — it is a consequence of the symbol-space reduction invariant. The algorithm cannot be made to run deeper by choice of input.

4.2 Losslessness

Theorem 2 (AIM is Lossless). *For any byte sequence D, decode(encode(D)) = D.*

Proof sketch. Each recursive call stores the selected bit b^*, the flag set F, and the encoded child (which encodes the remapped aligned stream). Reconstruction reads F, decodes the child to recover the remapped aligned stream, applies unremap to recover A, then sets bit b^* for each position in F. This inverts bit_clear exactly. By induction on depth: the base case (HALT_ZERO, HALT_ONE, HALT_DEPTH) is trivially invertible; the inductive case follows from the bijectivity of remap/unremap and the reversibility of flag set application. ∎

4.3 Parameter Freedom

AIM has no tunable parameters. The bit selection is determined by the data (argmin sweep). The format selection is determined by output size (race). The halt conditions are structural. The recursion depth is bounded by the alphabet size. This is the precise sense in which AIM is *parameter-free*: every decision is a function of the data and the algorithm, not of any externally specified constant.

This property matters for the Huffman analogy. Huffman coding is not merely a good compression technique — it is foundational because it is optimal for its problem class (minimum expected code length for known symbol distributions) without external parameters. AIM claims the analogous position for bit-plane structure: it is the natural, parameter-free, deterministically-terminating operation on this class of structure.

Whether it is *optimal* for that class in an information-theoretic sense is a question we leave for future work. The claim here is completeness and universality, not optimality.

4.4 Relationship to Kolmogorov Complexity

A complete AIM encoding — the selected bits, flag format IDs, compressed flag sets, and halt codes at each depth — constitutes a valid self-sufficient reconstruction program for the original data. No external dictionary, pre-trained model, or shared context is required. The output is complete in itself.

Any encoding improvement — a better flag format, a more efficient remap — reduces the length of this program. Because the algorithm is universal (it applies to arbitrary byte sequences), such improvements apply to all data simultaneously. This is the precise sense in which AIM is a computable approach toward Kolmogorov complexity: not by solving the non-computable optimum, but by defining a well-formed search space in which progress is measurable and improvements are universal.

5. Empirical Evaluation

5.1 Test Suite

All results are reported against gzip level 9 (raw H0 entropy coding as baseline). The test suite covers eight synthetic cases designed to probe different structural regimes:

File	Size	Description	AIM result
all_zeros	10 KB	Single byte value 0x00	HALT_ZERO at depth 0: 0.56% of raw H0
all_0xff	10 KB	Single byte value 0xFF	HALT_ONE at depth 0: 1.52% of raw H0
uniform_byte	10 KB	Repeated single non-zero/one byte	Structural collapse in 1–2 depths
low_alphabet	30 KB	Values in range [0, 7] (3-bit)	Rapid collapse: 0.66% of H0
mod7	50 KB	Bytes distributed mod 7	0.56% of H0
ramp_100k	100 KB	All 256 byte values × 400	0.47% of H0 (maximally flat)
repeated_text	12 KB	ASCII text phrase repeated × 1000	1.61% of H0
source_file	48 KB	Python source code (aim_v14.py)	65.82% of H0 (incompressible by structure)

5.2 Primary Benchmark: PCM Audio

The primary empirical result is compression of *sample-full.wav*, a 40,581,228-byte real-world PCM audio file (38.7 MiB). This file type was selected because its structural properties — interleaved high and low bytes of 16-bit samples with markedly different entropy profiles — are precisely the regime where AIM's bit-plane separation should provide the largest advantage over sliding-window compression.

```
$ ./aim encode sample-full.wav sample.aim4

Encoded  'sample-full.wav'  (40581228 bytes)
  Mode     : recursive
  Output   : 34482879 bytes  (84.97%)
  Raw H0   : 40581488 bytes  (100.00%)
  Delta    : -6098609 bytes  (-15.03%)
  Written  : 'sample.aim4'  [7.26s]
```

The −15.03% result is a structural improvement, not a benchmark artifact. The mechanism is explicable from first principles:

- **Bit-plane asymmetry in PCM audio.** 16-bit PCM samples store amplitude as a signed integer. The high byte (sample magnitude) clusters near zero during silence and soft passages; the low byte (fractional) is near-uniform regardless of signal level. The entropy profiles of these two bytes are completely different. Any encoding that treats them identically — as a sliding window compressor must — is conflating two structurally unlike streams.
- **Recursive separation.** AIM's sweep at depth 0 finds the sparsest bit plane across the full stream — typically a high-order bit in the high byte for audio with significant silence. Separating this plane reduces the entropy of the aligned stream, which in turn makes deeper bit planes cheaper to separate. The recursion amplifies the initial structural advantage.
- **Flag set compressibility.** The positions of set bits in the high byte of PCM audio are not random — they correspond to the onset and offset of acoustic events. This positional regularity is exactly what LZ77 and LZ77+Huffman in the flag format race exploit. The flag sets are smaller than their information-theoretic lower bound under the uniform assumption.

The result was derived analytically before the C implementation was run. The C binary confirmed the prediction exactly.

5.3 Implementation Comparison

Metric	Python (aim_v14.py)	C (aim.c)
Encode time (38.7 MB WAV)	~114 seconds	7.26 seconds
Compression ratio (WAV)	−15.03%	−15.03%
Encode time (10 MB synthetic)	~35 seconds	5.5 seconds
Decode + verify (10 MB)	~8 seconds	0.74 seconds
C → C roundtrip (11 test cases)	—	11/11 SHA-256 verified
Python → C decode	—	11/11 SHA-256 verified
C → Python decode	—	11/11 SHA-256 verified

The C implementation is approximately 15× faster than Python on large files. Compression ratios are identical to four significant figures across all tested inputs. The two implementations are wire-compatible: any file encoded by one can be decoded by the other.

6. The Huffman Analogy

Huffman coding finds the shortest possible prefix code given a known symbol distribution. Its contribution was foundational because it is universal over symbol distributions, lossless, and deterministic — and because it formalized a space of structure (symbol frequency) that prior methods had addressed only heuristically.

Before Huffman, frequency-aware encoding existed. Fixed codebooks, Morse code, and similar schemes exploited symbol frequency. What Huffman provided was the *optimal parameter-free algorithm* for that class of structure — one that requires no external constants, applies to any distribution, and produces the provably shortest expected code length.

AIM claims the analogous position one level of abstraction higher:

Property	Huffman	AIM
Structure class	Symbol frequency	Bit-plane sparsity
Operation	Variable-length prefix codes	Recursive bit-plane separation with remapping
Parameters	None	None
Terminates?	Yes (tree construction)	Yes (alphabet halves per depth, max 8)
Lossless?	Yes	Yes
Relationship	Optimal for its class	Universal for its class (optimality: open)
Prior work	Heuristic codebooks	Domain-specific bit-plane coders

The analogy is precise in one important respect: Huffman does not replace sliding-window compression, and AIM does not replace Huffman. In the current AIM implementation, Huffman coding is one of the seven formats in the flag format race. The two techniques are orthogonal — they operate on different structural dimensions of the data. A complete compression pipeline would apply both.

The claim is not that AIM is the solution to lossless compression. The claim is that *if a general method for approaching Kolmogorov complexity exists, it probably has this shape*: parameterized, recursive, lossless, deterministically halting.

7. Wire Format

The AIM container format is self-describing. A decoder requires no external context, shared dictionary, or pre-trained model.

7.1 Container Header (45 bytes)

Field	Size	Value
Magic	4 bytes	'AIM4'
Mode	1 byte	0 = recursive, 1 = CAIM
orig_n	8 bytes	uint64 big-endian: original byte count
SHA-256	32 bytes	SHA-256 of original data (integrity check)

7.2 Recursive Level Header (11 bytes + flag data)

Field	Size	Value
bit	1 byte	Selected bit position b* (0–7)
fmt	1 byte	Flag format ID (0–6, see Section 3.2)
flag_len	4 bytes	uint32 big-endian: byte length of flag data
halt	1 byte	0=recurse, 1=terminal, 2=HALT_ZERO, 3=HALT_ONE
child_len	4 bytes	uint32 big-endian: byte length of child payload
flag_data	flag_len bytes	Compressed flag set in format fmt

7.3 C-AIM Payload

```
n_levels      (1 byte)  : number of depths processed
bits[n]       (n bytes) : selected bit b* per depth
flags_len     (4 bytes) : uint32 BE: byte length of concatenated flags
flags_gz      (var)     : gzip of concatenated flag bitsets
term_len      (4 bytes) : uint32 BE: byte length of terminal gzip
term_gz       (var)     : gzip of final aligned stream
```

8. Discussion

8.1 The Underutilization of Bit-Plane Structure

The principle that numerical representations carry structure compressible independently of semantic content appears in fragments across bit-plane coding, predictive coding, and wavelet analysis. What has not been done systematically is to apply this principle universally — without domain knowledge, without a prior transform — and to treat the shape of the resulting decomposition as informative in its own right.

AIM's decay profile — the curve showing how much entropy remains at each depth, with the bit distribution per layer — is a structural fingerprint of the data that Shannon entropy does not capture. Two datasets with identical Shannon entropy can have completely different decay profiles. Random noise and PCM audio are both high-entropy byte sequences. AIM distinguishes them precisely, in a way that is predictive of how well a given compression strategy will perform.

8.2 Limitations

AIM does not improve on gzip for data whose structure is primarily sequential and positional (natural language text, source code). On these types, the flag sets produced by bit-plane separation are not significantly more compressible than the original, and the per-level overhead reduces the net ratio.

The current flag format race does not include arithmetic coding or ANS. Adding them would improve the ratio on flag sets with non-uniform symbol distributions — a direction for future work.

8.3 Future Work

- Arithmetic coding / ANS in the flag format race for flag sets in the 15–50% density regime.
- Corpus-level flag encoding: flag layers from structurally similar files share distribution properties, making cross-file deduplication potentially significant.
- Optimality characterisation: is AIM optimal for the bit-plane structure class in the information-theoretic sense, or is there a parameter-free recursive algorithm that achieves strictly better expected compression on all bit-plane-structured inputs?
- Generalized targets: the current implementation uses bit-clearing ($b^* \in \{0,\ldots,7\}$). The AIM formula supports arbitrary valid targets — mod-N alignment, bitwise masking — that may find additional structure in quantized physical measurement data.
- Entropy stratigraphy as a data identification tool: the decay profile is a structural fingerprint. A reference library of profiles for known data types would allow AIM to be used as a format-agnostic classifier.

9. Conclusion

We have presented Adaptive Isolating Model (AIM), a parameter-free, lossless, deterministically-terminating algorithm for recursive bit-plane decomposition of arbitrary byte data. The algorithm requires no training data, no shared context, no external dictionary, and no tunable constants. Every decision — which bit to separate, which codec to use for the flag set, when to halt — is a function of the data and the algorithm.

The central result is −15.03% versus gzip on a 40.6 MB real-world PCM audio file. This number was derived analytically before the C implementation was run and was confirmed exactly on first execution. It is not a benchmark artifact — it is the direct consequence of AIM's ability to access bit-plane structure that no sliding-window compressor can reach, explained completely by the algorithm's properties.

The theoretical contribution is the termination proof: the symbol space halves at each remap, bounding recursion at eight depths for byte data without any explicit depth limit parameter. This proof is what elevates AIM from a useful heuristic to a formally characterized algorithm. The halt conditions are structural facts, not engineering choices.

The Huffman analogy, stated precisely: Huffman formalized the optimal parameter-free algorithm for symbol-frequency structure. AIM claims the analogous position for bit-plane structure — a class of structure that is present in all byte data, exploited only in domain-specific forms by existing methods, and accessible universally by the sweep-and-clear architecture. Whether AIM is *optimal* for that class remains an open question.

That it is *universal* and *parameter-free* is established here.

The goal is not to beat gzip. The goal is to know what you are compressing.

. References

Ziv, J. and Lempel, A. (1977). *A universal algorithm for sequential data compression.* IEEE Transactions on Information Theory, 23(3):337–343.

Ziv, J. and Lempel, A. (1978). *Compression of individual sequences via variable-rate coding.* IEEE Transactions on Information Theory, 24(5):530–536.

Huffman, D. A. (1952). *A method for the construction of minimum-redundancy codes.* Proceedings of the IRE, 40(9):1098–1101.

Rissanen, J. and Langdon, G. G. (1979). *Arithmetic coding.* IBM Journal of Research and Development, 23(2):149–162.

Duda, J. (2009). *Asymmetric numeral systems.* arXiv:0902.0271.

Deutsch, P. (1996). *DEFLATE compressed data format specification version 1.3.* RFC 1951, IETF.

Mallat, S. G. (1989). *A theory for multiresolution signal decomposition: the wavelet representation.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 11(7):674–693.

Taubman, D. S. and Marcellin, M. W. (2002). *JPEG 2000: Image Compression Fundamentals, Standards and Practice.* Kluwer Academic Publishers.

Kolmogorov, A. N. (1965). *Three approaches to the quantitative definition of information.* Problems of Information Transmission, 1(1):1–7.

Li, M. and Vitányi, P. (2008). *An Introduction to Kolmogorov Complexity and Its Applications.* Springer, 3rd edition.

Robinson, T. (1994). *SHORTEN: Simple lossless and near-lossless waveform compression.* Technical Report CUED/F-INFENG/TR.156, Cambridge University Engineering Department.

Welch, T. A. (1984). *A technique for high-performance data compression.* Computer, 17(6):8–19.

. Appendix A: Availability

The following materials are provided as supplementary deliverables:

Artifact	Description
aim.py	Python reference implementation. All 7 flag formats, recursive and CAIM modes, full test suite. No dependencies beyond Python standard library and zlib.
aim.c	C reference implementation. Single file, 1355 lines. Dependency: zlib only. Build: gcc -O3 -o aim aim.c -lz -lm
AIM_Specification.pdf	Complete wire format specification. Sufficient to implement a correct AIM decoder in any language without reference to these source files.

Both implementations are wire-compatible. A file encoded by the Python implementation decodes correctly with the C implementation and vice versa, verified across all test cases with SHA-256 integrity checking.

. Appendix B: Flag Format Detail

Format	ID	Wire structure	Applicability
GAP	0	first_pos (varbyte) + successive_gaps (varbyte each)	Sparse: density < ~5%
BITSET	1	dense bit vector, MSB first, length = ceil(n/8) bytes	General; always applicable
ELIAS-FANO	2	EF encoding of sorted integer sequence	Monotone with clustered gaps
RLE	3	Run-length encoding on bitset representation	Alternating runs
HUFFMAN	4	lengths[256] + n_bits (4B BE) + packed bits MSB-first	Moderate density with byte repetition
LZ77	5	orig_n (4B) + n_tokens (4B) + flag-grouped token stream	Positional repetition in flag stream
LZ77+HUFF	6	LZ77 tokens Huffman-coded (= DEFLATE / gzip)	Large flag sets with mixed structure

The race is exhaustive: all seven codecs encode every flag set, and the shortest result is stored. On typical inputs at typical depths, the winner is LZ77+HUFFMAN for flag sets above ~50KB, BITSET or HUFFMAN for mid-range, and GAP for sparse sets below 5% density. The race overhead is negligible relative to the compression operation itself.